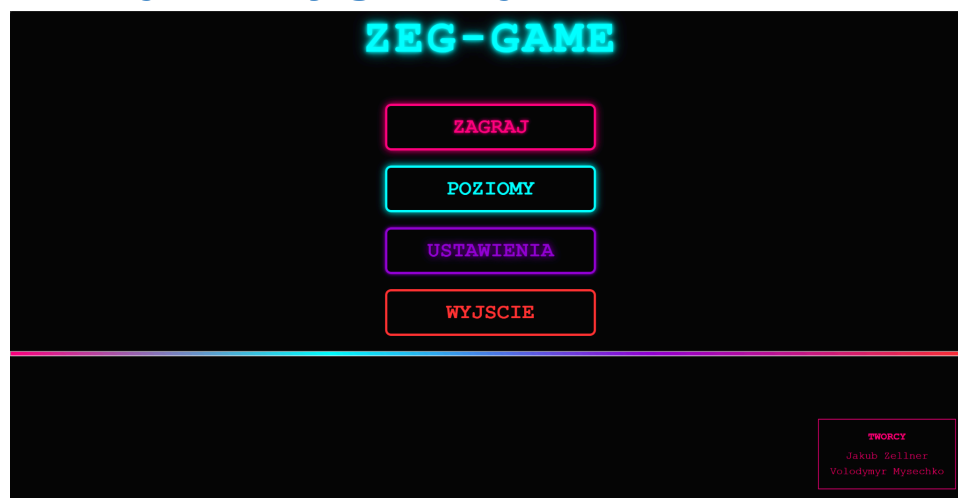


Interfejs strony głównej



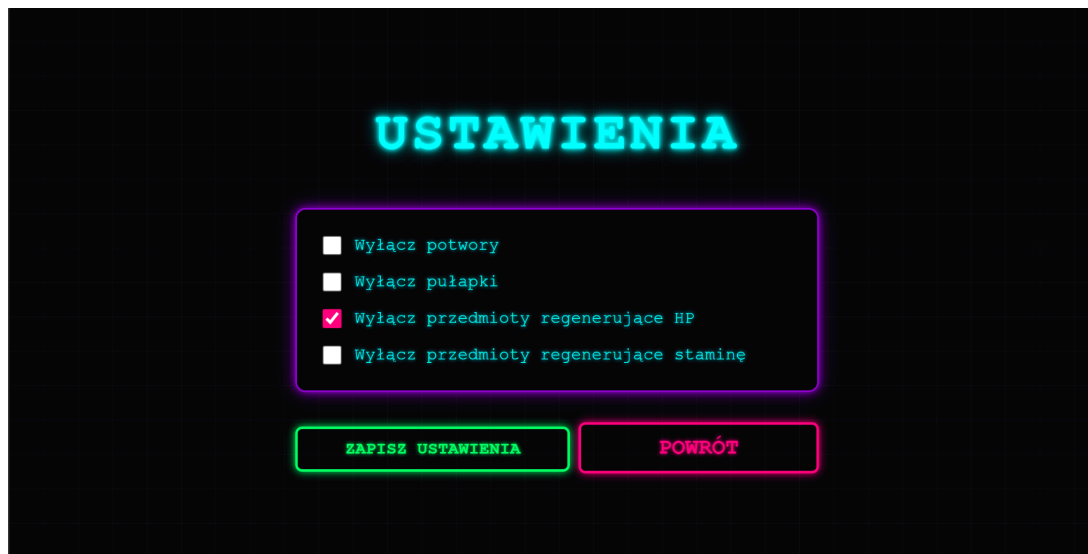
Interfejs poziomów



Interfejs strony z wyborem poziomów



Interfejs ustawien



Zespół Programistyczny: Volodymyr Mysechko, Jakub Zellner

Tytuł projektu : ZEG-GAME

Czas realizacji : do 08.06.2026

1. Ogólny opis projektu

1.1. Cel projektu

W środowisku przeglądarki internetowej stworzono nowoczesną grę typu **Labirynt**. W trakcie rozgrywki gracz unika zagrożeń, rozwiązuje zagadki logiczne, zbiera przedmioty oraz steruje postacią poruszającą się po planszy. Do wykonania projektu wykorzystano technologie JavaScript oraz HTML5 Canvas, a cały proces realizacji przebiegł zgodnie z metodyką Kanban.

1.2. Charakterystyka gry

Gra oferuje kilka etapów, których poziom trudności sukcesywnie wzrasta. W skład każdego poziomu wchodzi następujące elementy:

- labirynt
- przeciwnicy
- pułapki
- przedmioty możliwe do zebrania
- zagadki logiczne
- punkt końcowy, który umożliwia przejście do kolejnego etapu.

Gracz rozpoczyna rozgrywkę z pulą 100 punktów życia oraz 100 punktami staminy. Na mapie rozmieszczone są przedmioty, które podtrzymują balans rozgrywki i umożliwiają przejście labiryntu: pioruny odnawiają dokładnie 30 punktów staminy, natomiast serduszka regenerują 25 punktów życia. W przypadku kolizji z przeszkodami lub przeciwnikami gracz traci 10 punktów życia.

2. Wymagania funkcjonalne

2.1. Sterowanie postacią

- Poruszanie klawiszami WASD oraz strzałkami.
- Wykrywanie kolizji ze ścianami oraz blokowanie przejścia przez ściany.
- Wykrywanie kolizji z zagadkami oraz zapobieganie wyjściu z zagadki, dopóki nie zostanie rozwiązana.
- Płynne przejście między polami.

2.2. Mechanika labiryntu

- Plansza oparta na siatce.
- Start i koniec.
- Wykrywanie kolizji ze ścianami.
- Ściany i przejścia.
- Przechodzenie pomiędzy poziomami.

2.3. System przedmiotów

- Dostępne przedmioty:
 - Bonusy HP.
 - Bonusy staminy.
 - Aktywatory zagadek.
- Funkcjonalności:
 - Znikanie przedmiotów po podniesieniu.
 - Aktualizacja paska staminy.
 - Aktualizacja paska HP.

2.4. Zagadki logiczne

- Typy zagadek:
 - Pytania logiczne.
 - Pytania powiązane z programowaniem lub podanym kodem.
 - Proste działania matematyczne.
- Funkcjonalność:
 - Blokowanie przejścia, dopóki zagadka nie zostanie odgadnięta.
 - Wyświetlanie komunikatów o poprawnej lub błędnej odpowiedzi.
 - Sprawdzenie poprawności odpowiedzi.

2.5. System przeciwników

- Potwory:

- Przemieszczają się po mapie z prędkością dostosowaną do ukształtowania labiryntu oraz do prędkości gracza.
- Zadają obrażenia zmniejszające HP gracza.
- Wykrywanie kolizji z graczem.
- Pułapki:
 - Nieruchome kapkany przeszkadzające w kontynuowaniu przejścia.
 - Wykrywanie kolizji z graczem.
 - Utrata HP po kontakcie.

2.6. System poziomów

- 5 poziomów.
- Zwiększenie trudności.
- Na każdym kolejnym poziomie rozmiar labiryntu się zwiększa.
- Na każdym poziomie jest więcej potworów i pułapek.
- Trudniejsze zagadki.

2.7. Interfejs

- Pasek HP.
- Pasek staminy.
- Aktualny poziom.
- Informacja o przejściu do następnego poziomu, przegranej lub przejściu całej gry.
- Ekran zagadki.

2.8. Funkcje dodatkowe

- Przyciski na dole każdego poziomu służące do łatwiejszego przeglądania poziomów.
- Możliwość restartu gry.

3. Wymagania niefunkcjonalne

3.1. Wydajność

- Płynne działanie gry.
- Szybkie ładowanie poziomów.
- Lekkie zasoby graficzne.

3.2. Użyteczność

- Prosty, przejrzysty interfejs.
- Czytelny układ.
- Komunikaty.
- Intuicyjne sterowanie.
- Wygląd gry w jednakowym stylu.

3.3. Responsywność

- Gra poprawnie działa na komputerach oraz laptopach.
- Obsługa różnych rozdzielczości ekranu.

3.4. Bezpieczeństwo i stabilność

- Zabezpieczenie przed błędami JavaScript.
- Kontrola poprawności stanu gry.

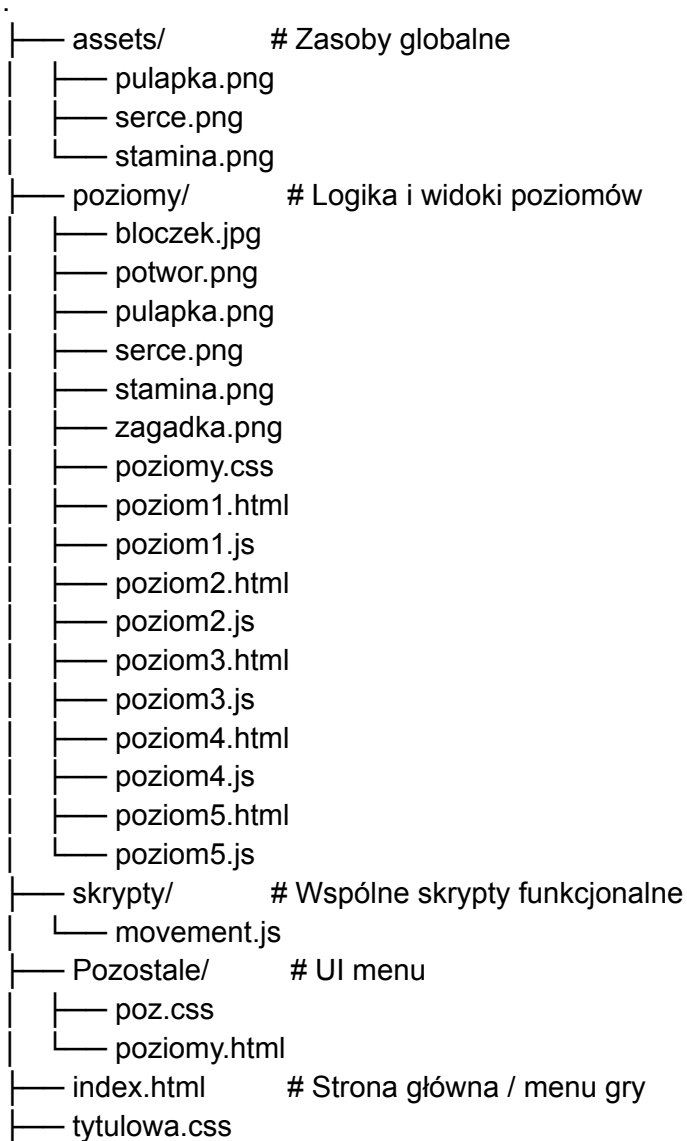
3.5. Interfejs menu (strony głównej)

W menu znajdują się 4 przyciski:

1. „ZAGRAJ” – rozpoczyna grę, zaczynając od 1. poziomu.
2. „Poziomy” – przekierowuje na stronę z 5 przyciskami, po kliknięciu których zostaniemy przekierowani na wybrany poziom.
3. „Ustawienia” – przekierowuje do strony z ustawieniami, gdzie mamy możliwość wyłączenia pojawiających się bonusów na mapie (HP, pioruny).
4. „Wyjście” – zamyka stronę.

4. Struktura projektu

4.1. Struktura plików



4.2. Opis modułów

- **Assets:** Zasoby globalne; zawiera zdjęcia pułapki, serca, piorunu.
- **Poziomy:** Folder z 5 poziomami.
 - `poziom1.js` – `poziom5.js`: siatka, rozmieszczenie przeciwników, pułapek, zagadek na danym poziomie.
 - `poziomy.css`: zawiera styl; dla wszystkich poziomów jest taki sam.
 - Folder ten zawiera również szkielet HTML dla każdego poziomu osobno oraz zasoby graficzne, np. zdjęcie potwora, pytajnik (zagadka).
- **Skrypty:**
 - `movement.js` stanowi centralny moduł odpowiedzialny za mechanikę rozgrywki. Odpowiada za obsługę stanu gry oraz interakcje postaci z otoczeniem. Do kluczowych zadań tego modułu należą:
 - Logika przemieszczania: obsługa sterowania postacią oraz wykrywanie kolizji z elementami otoczenia (ścianami).
 - Zarządzanie stanem gracza: monitorowanie i aktualizacja puli punktów życia (HP) oraz punktów staminy w czasie rzeczywistym.
 - System bonusów i zagrożeń: obsługa interakcji z obiektami na mapie, w tym regeneracja zasobów (pioruny: +30 staminy, serduszka: +25 HP) oraz naliczanie obrażeń w przypadku kolizji z przeciwnikami lub pułapkami (-10 HP).
 - Zarządzanie etapami: odnajdywanie punktów startowych i wyjściowych na planszy oraz reagowanie na pomyślne przejście labiryntu.
 - Renderowanie: odpowiada za proces rysowania gracza w aktualnej pozycji na płótnie gry.
 - Logika zagadek: także odnajduje obecny poziom i przekierowuje na następny.
- **Pozostałe:**
 - `poziomy.html`: szkielet strony z nawigacją po poziomach.
 - `poz.css`: stylizacja strony `poziomy.html`.
 - `ustawienia.html`: strona z ustawieniami używana do wyłączania bonusów.
 - `ustawienia.js`: logika działania ustawień.
 - `ustawienia.css`: stylizacja ustawień.

5. Dokumentacja kodu

5.1. Komentarze

Kod został opisany komentarzami, żeby:

- polepszyć czytelność,
- ułatwić dalszy rozwój,
- szybciej lokalizować błędy.

poziom1.js - poziom5.js

```
const plotno = [  
  [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],  
  [0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1],  
  [1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1],  
  [1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1],  
  [1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1],  
  [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],  
  [1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1],  
  [1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1],  
  [1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1],  
  [1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1],  
  [1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1],  
  [1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1],  
  [1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1],  
  [1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0],  
  [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]  
];
```

Mapa jest zdefiniowana jako dwuwymiarowa tablica, która stanowi fundament labiryntu. W tej strukturze wartość 1 oznacza ścianę, natomiast 0 oznacza wolną przestrzeń.

```
const zagadki = [
  {row:13,col:1, pytanie: "Jak nazywa się pojedynczy znak tekstu w języku C++", odp: "char"}]
```

Tablica zagadki przechowuje obiekty definiujące wyzwania logiczne umieszczone na mapie. Każdy obiekt zawiera współrzędne położenia (row, col) oraz przypisaną treść pytania wraz z prawidłową odpowiedzią.

```
const bonusy = [
  { row: 3, col: 1, typ: "hp" },
  { row: 9, col: 7, typ: "stamina" }]
```

Tablica bonusy przechowuje obiekty definiujące przedmioty wspomagające gracza rozmieszczone na mapie. Każdy obiekt zawiera współrzędne położenia (row, col) oraz określenie typu bonusu (hp lub stamina), co pozwala na ich odpowiednią identyfikację i renderowanie.

```
const zagrozenia = [
  { row: 3, col: 5, typ: "pulapka" },
  {row:23,col:7,speed:0.06,typ:"prawo-lewo",dir:1,startCol:5,endCol:11}]
```

Tablica zagrozenia przechowuje obiekty definiujące niebezpieczeństwa rozmieszczone na mapie. Każdy obiekt zawiera współrzędne położenia (row, col) oraz typ zagrożenia:

- **Pułapki:** Obiekty typu "pulapka" reprezentują statyczne zagrożenia zajmujące konkretne pole na planszy.
- **Przeciwnicy ruchomi:** Obiekty z określonym typem ruchu (np. "prawo-lewo") zawierają dodatkowe parametry, takie jak prędkość (speed), kierunek (dir)

oraz granice ruchu (startCol, endCol), które umożliwiają automatyczną animację postaci wzdłuż wyznaczonej ścieżki.

```
function updateMonsters() {
  zagrozenia.forEach(m => {
    if (m.typ === "ruchomy") {
      m.col += m.dir * m.speed;

      if (m.col >= m.endCol) {
        m.col = m.endCol;
        m.dir = -1; // lewo
      } else if (m.col <= m.startCol) {
        m.col = m.startCol;
        m.dir = 1; // prawo
      }
    }
  });
}
```

!!!WAŻNE!!!

-Powyższa funkcja działa tylko na poziomie 1, na pozostałych poziomach jest zrobiona w inny sposób

Funkcja updateMonsters odpowiada za logikę ruchu przeciwników typu “ruchomy”. Działanie funkcji obejmuje:

- Przeszukanie typów obiektów dopóki w poszukiwaniu z typem “ruchomy”
- W każdym wywołaniu funkcja przesuwa przeciwnika o określoną wartość w kierunku zdefiniowanym przez zmienną dir
- Logika zmiany kierunku: Jeśli przeciwnik osiągnie prawy punkt graniczny (endCol), funkcja blokuje jego pozycję na tej wartości i zmienia kierunek na przeciwny (dir = -1, ruch w lewo).
Jeśli przeciwnik osiągnie lewy punkt graniczny (startCol), funkcja blokuje jego pozycję na tej wartości i zmienia kierunek na przeciwny (dir = 1, ruch w prawo).

Poniższa funkcja jest stosowana w poziomach 2-5

```
function updateMonsters() {
  zagrozenia.forEach(m => {
    if (m.typ === "gora-dol") {
      m.row += m.dir * m.speed;

      if (m.row >= m.endRow) {
        m.row = m.endRow;
        m.dir = -1;
      } else if (m.row <= m.startRow) {
        m.row = m.startRow;
        m.dir = 1;
      }
    }
    else if (m.typ == "prawo-lewo") {
      m.col += m.dir * m.speed;

      if (m.col >= m.endCol) {
        m.col = m.endCol;
        m.dir = -1; // w lewo
      } else if (m.col <= m.startCol) {
        m.col = m.startCol;
        m.dir = 1; // w prawo
      }
    }
  });
}
```

- Ruch pionowy (typ:"gora-dol")
- Wiersz (m.row) przeciwnika jest aktualizowany o wartość wynikającą z kierunku (dir) oraz prędkości.
- W przypadku osiągnięcia dolnej granicy (endRow), kierunek zmienia się na ujemny (ruch w górę).

- W przypadku osiągnięcia górnej granicy(startRow), kierunek zmienia się na dodatni (ruch w dół).

Ruch poziomy (typ:"prawo-lewo"):

- Kolumna przeciwnika jest aktualizowana o wartość wynikającą z kierunku (dir) oraz prędkości(speed).
- W przypadku osiągnięcia prawej granicy(endCol), kierunek zmienia się na ujemny (ruch w lewo).
- W przypadku osiągnięcia lewej granicy(startCol), kierunek zmienia się na dodatni (ruch w prawo).

```
function rysuj() {
```

Funkcja `rysuj()` odpowiada za poprawne wyświetlenie wszystkich elementów gry na płótnie w każdej klatce animacji. Jej działanie można podzielić na sekcje odpowiadające za poszczególne warstwy wizualne:

- **Renderowanie mapy:** Funkcja iteruje po całej siatce mapy, wypełniając tło kolorem czarnym. W miejscach, gdzie w tablicy mapy znajduje się ściana, rysowany jest odpowiedni bloczek graficzny, a wokół każdego pola rysowana jest obwódka.
- **Wyświetlanie zagadek:** Wszystkie obiekty reprezentujące zagadki logiczne są nanoszone na planszę jako ikony, pozostając widoczne dla gracza przez cały czas trwania rozgrywki.
- **Obsługa zagrożeń:** Funkcja sprawdza typ oraz ustawienia globalne (włączenie lub wyłączenie pułapek i potworów). Jeśli dany element jest aktywny, rysuje pułapki w ich pozycjach lub animowane postacie potworów, skalując ich rozmiar dla uzyskania lepszego efektu wizualnego.
- **Renderowanie bonusów:** Na podstawie bieżących ustawień widoczności, funkcja nanosi na mapę serduszka regenerujące życie oraz pioruny regenerujące staminę, o ile gracz nie wyłączył ich wyświetlania w panelu ustawień.

movement.js

```
const gracz = {

  // pozycja
  x: 0,
  y: 0,

  // rozmiar gracza
  width: size * 0.5,
  height: size * 0.5,

  // prędkość
  speed: 1.2,
  sprintSpeed: 2.0,

  // stamina
  stamina: 100,
  maxStamina: 100,

  staminaDrain: 0.65,
  staminaRegen: 0.15,

  //opóźnienie ładowania wytrzymałości
  staminaDelay: 3000,
  lastSprintTime: 0,

  // zdrowie
  hp: 100,
  maxHp: 100,

  damageCooldown: 2000,
  lastDamageTime: 0
};
```

Moduł gracza (gracz)

- **Pozycja i wielkość:** Informacja, gdzie postać znajduje się na mapie i jak duża jest.
- **Ruch:** Prędkość zwykłego chodu oraz szybszego sprintu.
- **Stamina :** Ile energii ma gracz, jak szybko się ona kończy podczas biegu i jak powoli odnawia się po przerwie.
- **Zdrowie (HP):** Ile życia ma postać. Dodatkowo jest tu licznik, który sprawia, że gracz nie traci całego życia od razu, jeśli wpadnie w pułapkę kilka razy w bardzo krótkim czasie.

```
function ustawStart() {  
  
    for (let row = 0; row < plotno.length; row++) {  
  
        // szuka wejścia po lewej  
        if (plotno[row][0] === 0) {  
  
            // spawn wewnątrz mapy  
            gracz.x = size * 0.2;  
  
            // środek kratki  
            gracz.y =  
                row * size +  
                (size / 2) -  
                (gracz.height / 2);  
  
            return;  
        }  
    }  
}
```

Funkcja ustawiania startu (ustawStart)

Ta funkcja decyduje, w którym miejscu na początku gry pojawi się nasza postać. Działa w ten sposób:

- **Szukanie wejścia:** Przeszukuje lewą krawędź mapy , aż znajdzie puste pole, które oznacza wejście do labiryntu.
- **Ustawienie gracza:** Gdy znajdzie wolne miejsce, ustawia tam gracza.

```
function kolizja(x, y) {

    const left = Math.floor(x / size);

    const right = Math.floor((x + gracz.width) / size);

    const top = Math.floor(y / size);

    const bottom = Math.floor((y + gracz.height) / size);

    return (

        plotno[top]?.[left] === 1 || plotno[top]?.[right] === 1 ||
        plotno[bottom]?.[left] === 1 || plotno[bottom]?.[right] === 1

    );

}
```

Funkcja sprawdzania kolizji (kolizja)

- **Obliczanie narożników:** Funkcja sprawdza cztery rogi postaci . Dzięki temu gra wie dokładnie, czy którykolwiek fragment postaci dotyka ściany.
- **Wykrywanie przeszkody:** Jeśli w którymkolwiek z tych punktów na mapie znajduje się ściana (czyli wartość 1 w tablicy plotno), funkcja zwraca prawdę – oznacza to, że gracz nie może się tam poruszać.

```
function ruch() {
```

Funkcja poruszania się (ruch)

- **Sterowanie i prędkość:** Sprawdza, które klawisze są wciśnięte (klawisze WSAD lub strzałki) i oblicza, w którą stronę postać ma się przesunąć. Jeśli gracz przytrzyma Shift, postać zaczyna biec szybciej.
- **Zarządzanie staminą:** Podczas biegu poziom energii spada. Gdy gracz przestaje biegać, funkcja sprawdza, czy minęły 3 sekundy, a jeśli tak – powoli przywraca staminę do maksimum.
- **Sprawdzanie kolizji:** Zanim postać faktycznie się przesunie, funkcja sprawdza osobno dla osi X i osi Y, czy na nowej pozycji nie ma ścian. Jeśli droga jest wolna, dopiero wtedy aktualizuje pozycję gracza.
- **Ograniczenia mapy:** Na końcu funkcja pilnuje, aby postać nie wyszła poza obszar labiryntu .

```
function rysujHP() {
```

Funkcja rysowania paska zdrowia (rysujHP)

- **Tło paska:** Najpierw rysuje ciemnoszary prostokąt, który stanowi stałą podstawę paska, niezależnie od ilości zdrowia.
- **Poziom życia:** Rysuje na wierzchu jasno zielony prostokąt. Jego szerokość jest obliczana proporcjonalnie do obecnego stanu zdrowia gracza w stosunku do jego maksymalnego życia ($\text{aktualne HP} / \text{maksymalne HP} * \text{szerokość paska}$).
- **Ramka:** Na samym końcu rysuje białą obwódkę wokół całego elementu, aby pasek był wyraźnie widoczny na tle gry.

```
function sprawdzWyjście() {
```

Funkcja obsługi zakończenia poziomu (sprawdzWyjście)

Funkcja ta odpowiada za sprawdzenie, czy gracz dotarł do końca labiryntu i czy spełnił wszystkie warunki wymagane do przejścia dalej.

- **Weryfikacja pozycji:** Funkcja sprawdza, czy współrzędne gracza znajdują się na ostatniej kolumnie mapy oraz czy pole jest puste.
- **Kontrola postępów:** Sprawdzane jest, czy liczba rozwiązanych zagadek) jest równa wymaganemu limitowi .
 - **Jeżeli warunek jest spełniony:** Funkcja odczytuje nazwę aktualnego pliku, oblicza numer kolejnego poziomu i przenosi gracza do następnego etapu lub do menu głównego, jeśli gra została ukończona.
 - **Jeżeli warunek nie jest spełniony:** Wyświetlany jest komunikat z prośbą o rozwiązanie pozostałych zagadek, a gracz jest automatycznie cofany o jedną kratkę od wyjścia .

```
function sprawdzKolizjePotworow() {
```

Funkcja sprawdzania kolizji z potworami (sprawdzKolizjePotworow)

Funkcja ta jest odpowiedzialna za wykrywanie kontaktu gracza z zagrożeniami na mapie oraz zarządzanie utratą punktów życia.

- **Obliczanie obszaru kolizji:** Funkcja oblicza dokładną pozycję każdego przeciwnika, aby sprawdzić, czy jego pole styka się z polem zajmowanym przez gracza.
- **Wykrywanie kontaktu:** Sprawdzane jest, czy prostokąty wyznaczające granice gracza oraz przeciwnika na siebie nachodzą.
- **Zasada obrażeń:** Jeśli nastąpiło zetknięcie, funkcja sprawdza, czy od ostatniego otrzymania obrażeń minął wymagany czas. Zapobiega to utracie całego zdrowia w ułamku sekundy, gdy gracz stoi na polu przeciwnika.
- **Odejmowanie życia i zakończenie:** Jeśli warunki są spełnione, gracz traci 10 punktów życia. Jeżeli stan zdrowia spadnie do zera, wyświetlany jest komunikat o przegranej, a gra zostaje zresetowana.

```
function sprawdzZagadki() {
```

Oraz

```
window.sprawdzOdpowiedz = function() {
```

Obsługa zagadek (sprawdzZagadki oraz sprawdzOdpowiedz)

Te dwie funkcje odpowiadają za interakcję gracza z zagadkami logicznymi rozmieszczonymi na mapie:

- Aktywacja zagadki (sprawdzZagadki): Gdy gracz wejdzie na pole, na którym znajduje się zagadka, funkcja sprawdza współrzędne gracza. Jeśli się zgadzają, otwiera okienko z pytaniem, wstawia treść pytania do formularza.
- Weryfikacja odpowiedzi (sprawdzOdpowiedz):
 - Jeśli odpowiedź jest poprawna: Gra wyświetla gratulacje, zwiększa licznik rozwiązanych zagadek, zamyka okienko, czyści pole tekstowe i usuwa rozwiązana zagadkę z tablicy, aby nie pojawiała się ponownie.
 - Jeśli odpowiedź jest błędna: Gracz otrzymuje komunikat z prośbą o ponowną próbę, a okienko pozostaje otwarte.

```
function sprawdzBonusy() {
```

Funkcja zbierania bonusów (sprawdzBonusy)

Ta funkcja pozwala graczowi podnosić przedmioty leżące na mapie i korzystać z ich mocy. Działa w następujący sposób:

- **Wykrywanie kontaktu:** Funkcja sprawdza, czy pole zajmowane przez gracza jest tym samym polem, na którym znajduje się bonus.
- **Przyznawanie efektów:**
 - **HP :** Zwiększa zdrowie gracza o 25 punktów, pilnując jednocześnie, aby nie przekroczyło ono wartości maksymalnej.
 - **Stamina:** Dodaje 30 punktów do wytrzymałości, również dbając o to, by poziom energii nie przekroczył maksimum.
- **Usuwanie z planszy:** Po podniesieniu przedmiotu, funkcja usuwa go z tablicy bonusy, dzięki czemu znika on z ekranu i gracz nie może podnieść go ponownie.

```
function gameLoop() {
```

. Główna pętla gry (gameLoop)

To najważniejsza funkcja w całym kodzie, która działa jak „serce” gry.

- **Czyszczenie ekranu:** Usuwa poprzednią klatkę, aby przygotować czyste płótno na nową.
- **Rysowanie elementów:** Wyświetla mapę, bonusy, zagrożenia oraz postać gracza.
- **Aktualizacja logiki:** Przelicza ruch potworów, przesuwa gracza, sprawdza, czy gracz podniósł bonus lub wpadł na potwora oraz czy zbliżył się do zagadki.
- **Interfejs użytkownika:** Rysuje na ekranie paski staminy i zdrowia (HP), żeby gracz wiedział, w jakiej jest kondycji.
- **Kontrola końca:** Sprawdza, czy gracz dotarł do wyjścia.
- **Pętla:** Na samym końcu `requestAnimationFrame` mówi przeglądarce: "Kiedy będziesz gotowa, wykonaj `gameLoop` jeszcze raz", co tworzy płynną animację.

6.Zarządzanie projektem

6.1 Metodyka pracy

Projekt realizowany był zgodnie z metodyka Kanban

Narzedzia :

- Trello
- Github
- Visual studio code
- Figma

6.2 podział obowiązków

Zadanie główne	Podzadanie	Odpowiedzialny	Data rozpoczęcia	Data zakończenia
1. Analiza wymagań	Określenie celu gry i mechaniki	Volodymyr	2026-05-15	2026-05-15
	Wymagania funkcjonalne i techniczne	Volodymyr	2026-05-16	2026-05-16
2. Mapa i Siatka	Projekt map Level 1-5	Volodymyr	2026-05-17	2026-05-18
	Renderowanie siatki i ścian	Volodymyr	2026-05-19	2026-05-20
	Test przechodzalności mapy	Volodymyr	2026-05-21	2026-05-21
3. Ruch Postaci	Sterowanie klawiaturą (WASD)	Jakub	2026-05-17	2026-05-18
	Fizyka ruchu i animacja (sprite)	Jakub	2026-05-19	2026-05-21
4. System Kolizji	Wykrywanie kolizji ze ścianami	Volodymyr	2026-05-23	2026-05-24
	Blokowanie przejścia przez ściany	Jakub	2026-05-24	2026-05-25
5. Obiekty i Wrogowie	Grafiki przeciwników i animacje	Volodymyr	2026-05-26	2026-05-28
	Implementacja przedmiotów (Klucze/HP)	Volodymyr	2026-05-29	2026-05-30
	Rozmieszczenie obiektów na mapach	Volodymyr	2026-05-29	2026-05-30
6. Logika i UI	System zagadek logicznych	Volodymyr	2026-05-31	2026-06-01
	Integracja UI i ekranów menu	Jakub	2026-05-31	2026-06-02
7. Finalizacja	Testy końcowe i poprawki błędów	Zespół	2026-06-03	2026-06-04
	Dokumentacja końcowa PDF	Zespół	2026-06-05	2026-06-06

7. Testowanie aplikacji

7.1. Testowanie funkcjonalne

W celu zapewnienia poprawności działania wszystkich mechanizmów gry, przeprowadzono następujące testy funkcjonalne:

- **Sterowanie postacią:** Sprawdzono responsywność klawiatury oraz poprawność działania mechaniki sprintu i zużycia staminy.
- **Kolizje:** Zweryfikowano poprawność działania detekcji kolizji ze ścianami oraz z obiektami zagrożeń (pułapki, przeciwnicy).
- **Działanie przeciwników:** Przetestowano poprawność poruszania się potworów po wyznaczonych ścieżkach oraz ich wpływ na punkty życia gracza.
- **Poprawność poziomów:** Sprawdzono integralność mapy oraz warunki zakończenia poziomu w oparciu o liczbę rozwiązanych zagadek.
- **Działanie zagadek:** Zweryfikowano poprawne otwieranie okien dialogowych, logikę weryfikacji wpisanych odpowiedzi oraz usuwanie zagadki z gry po sukcesie.
- **Poprawność UI:** Przetestowano czytelność i aktualizację pasków stanu (zdrowie oraz stamina) w czasie rzeczywistym.

7.2. Testowanie wydajności

Przeprowadzono analizę wydajnościową w celu zapewnienia optymalnego działania gry:

- **Płynność animacji:** Potwierdzono stabilne działanie pętli gry (gameLoop) przy użyciu requestAnimationFrame, co zapewnia płynność ruchu bez zacięć.
- **Zużycie pamięci:** Zweryfikowano zarządzanie obiektami, w tym sprawne usuwanie elementów z tablic (np. rozwiązane zagadki, zebrane bonusy), co zapobiega wyciekom pamięci.

8. Możliwe błędy i ich przyczyny

Niepełne załadowanie zasobów gry: Czasami, przy bardzo szybkim otwarciu strony lub problemach z siecią, przeglądarka może próbować uruchomić gameLoop zanim wszystkie elementy mapy lub funkcje zostaną w pełni wczytane. W takim przypadku gra może nie działać poprawnie (np. brak gracza lub nieaktywna mapa). Rozwiązaniem jest odświeżenie strony, co wymusza ponowne, poprawne załadowanie wszystkich skryptów.